



PROGRAMAÇÃO



Escola Naval



António Gonçalves

AGENDA

Introdução à Informática



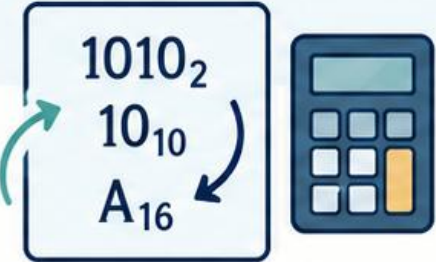
Arquitetura do computador

CPU, memória, armazenamento e periféricos.



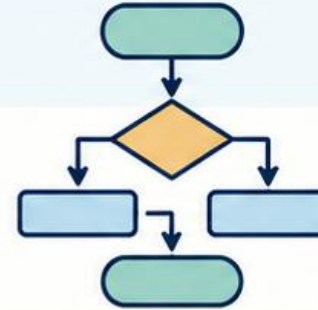
Representação da informação

Bits, bytes, ASCII e Unicode.



Sistemas de numeração

Binário, decimal e hexadecimal: conversões e adição.



Algoritmos

Entradas, processamento, saídas e decomposição de problemas.



Pseudocódigo e programas

Representar soluções e traduzí-las para uma linguagem.



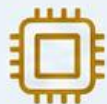
Aplicação e verificação

Resolver exercícios, testar resultados e corrigir erros.

ARQUITETURA DO COMPUTADOR

CPU: executa instruções e processa dados. Inclui unidade de controlo, unidade aritmética e lógica e registos.

RAM: guarda temporariamente programas e dados em utilização. É volátil.



Hardware: componentes físicos.



Software: programas e instruções.

Os componentes comunicam através de ligações e barramentos.

DA ABERTURA DE UM PROGRAMA AO RESULTADO

1. ARMAZENAR:

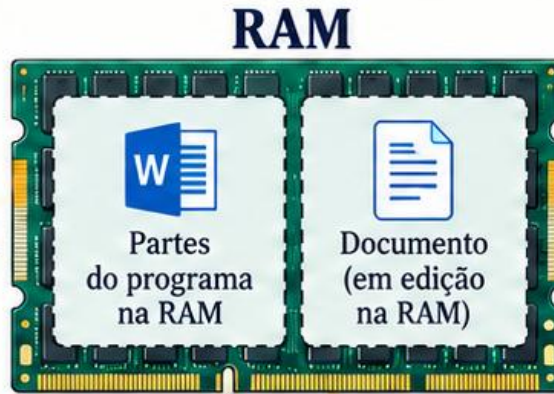
O programa e o documento estão guardados no SSD.



2. CARREGAR:

Ao abrir o programa, o sistema operativo carrega para a RAM as partes necessárias.

Partes necessárias do programa e do documento



3. EXECUTAR:

A CPU obtém e executa instruções, utilizando dados na RAM.

Instruções e dados

Resultados



Resultado gráfico



Texto digitado pelo utilizador



RAM: trabalho temporário.
SSD: conservação dos ficheiros.

5. GUARDAR:

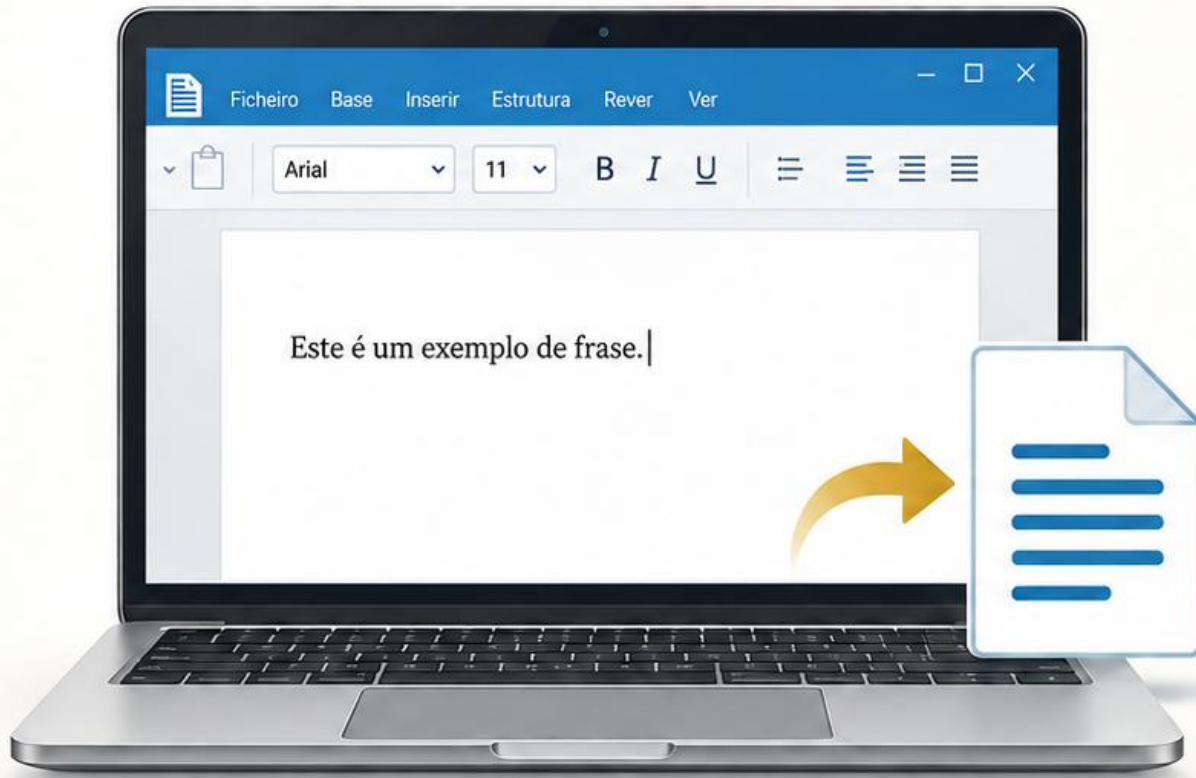
as alterações do documento são escritas no armazenamento.



Documento (guardado com as alterações)

PERGUNTA: COMPONENTES E FUNÇÕES

Abres um processador de texto, escreves uma frase e guardas o documento.



1. Onde está guardado o programa antes de o abrires?

2. Em que memória ficam as partes do programa em utilização?

3. Que componente executa as instruções?

4. O que acontece ao conteúdo da RAM quando desligas o computador?



Pensa individualmente. Justifica cada resposta com a função do componente.

RESPOSTA: COMPONENTES E FUNÇÕES

1 ARMAZENAMENTO:

O programa está guardado no SSD ou noutro dispositivo de armazenamento.



2 RAM:

Guarda temporariamente as instruções e os dados necessários à execução.



RAM
MEMÓRIA TEMPORÁRIA

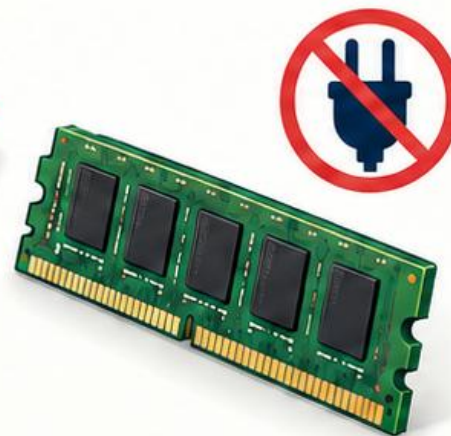
3 CPU:

Executa as instruções do programa.



4 RAM VOLÁTIL:

Sem alimentação elétrica, perde o seu conteúdo.



Programa
e dados

Instruções
e dados

Resultados



Um documento **guardado** permanece no SSD.

Alterações **ainda não guardadas** podem perder-se ao desligar.



Ficheiro guardado
(permanente)



Alterações não guardadas
(podem perder-se)

BITS, BYTES E BASES DE NUMERAÇÃO



1 byte = 8 bits

Bit: unidade de informação com dois valores possíveis, 0 ou 1.

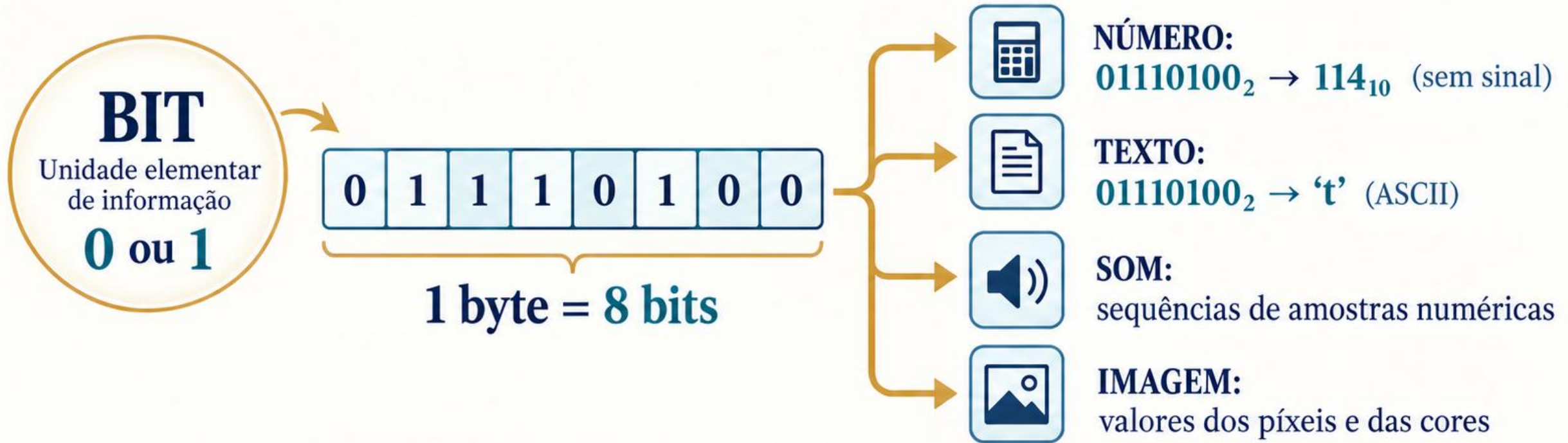
Sistema	Base	Símbolos	Exemplo
Decimal	10	0 a 9	13_{10}
Binário	2	0 e 1	1101_2
Hexadecimal	16	0 a 9 e A a F	D_{16}

$$13_{10} = 1101_2 = D_{16}$$

O índice indica a base. O valor é o mesmo; muda a representação.

Com 8 bits existem $2^8 = 256$ combinações.
Inteiros sem sinal: 0 a 255.

COMO O COMPUTADOR REPRESENTA DADOS



Como pode o mesmo padrão representar um número ou uma letra?



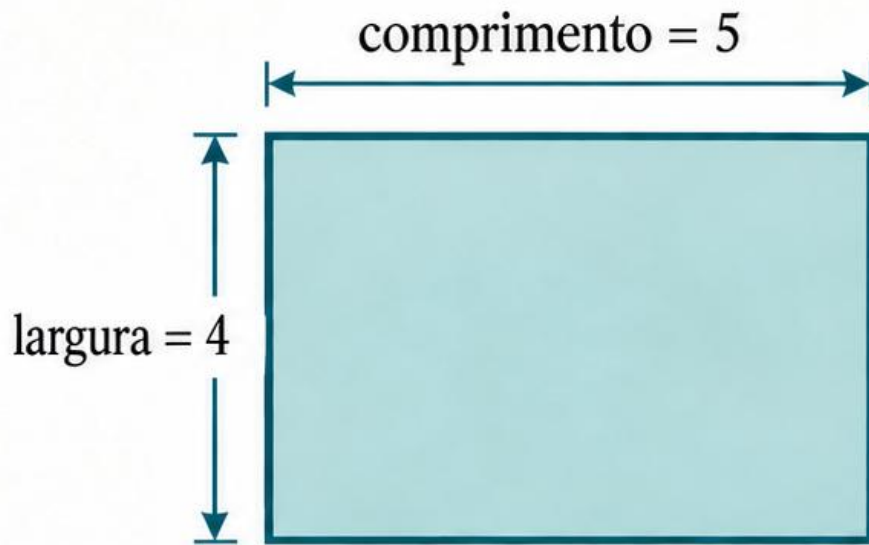
A interpretação depende do tipo de dados, da codificação e do software.



IDEIA-CHAVE: Os bits são os mesmos; o significado depende da interpretação.

O QUE É UM ALGORITMO?

Uma sequência finita de passos ordenados e precisos para resolver um problema.



- 1 Precisão:**
cada passo tem um significado claro.
- 2 Ordem:**
os dados são lidos antes do cálculo.
- 3 Terminação:**
o processo chega ao fim.



Uma conversão de base também segue um algoritmo:
recebe um número, aplica regras e produz outra representação.

ALGORITMO: BINÁRIO PARA DECIMAL

- 1 Numerar as posições da direita para a esquerda, começando em 0.
- 2 Atribuir a cada posição o peso 2 elevado à posição.
- 3 Multiplicar cada bit pelo seu peso.
- 4 Somar os produtos.

Os pesos duplicam para a esquerda



Posição:	7	6	5	4	3	2	1	0
Peso:	128	64	32	16	8	4	2	1

Valor decimal = soma de (**bit** × **peso**)

Bit 0: contribuição nula.

Bit 1: soma-se o peso.

EXEMPLO: 1011_2 EM DECIMAL

Posição	3	2	1	0
Peso	8	4	2	1
Bit	1	0	1	1
Contribuição	8	0	2	1

- 1 A posição mais à direita tem peso 1.
- 2 O bit 0 na posição de peso 4 não contribui para a soma.
- 3 O número 1011_2 representa onze, não mil e onze.

$$1011_2 = 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$

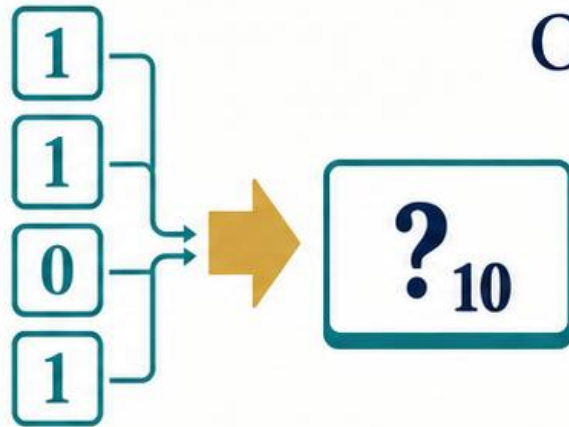
$\downarrow \quad \quad \downarrow \quad \quad \downarrow \quad \quad \downarrow$
 $8 \quad \quad 0 \quad \quad 2 \quad \quad 1$

$$1011_2 = 8 + 0 + 2 + 1 = 11_{10}$$

PERGUNTA: BINÁRIO PARA DECIMAL

$$1101_2 = ?_{10}$$

Converte o número e apresenta os cálculos.



Bit	1	1	0	1
Peso	_____	_____	_____	_____
Contribuição	_____	_____	_____	_____



Por que lado comesças a atribuir os pesos?



Que posições contribuem para a soma?



O zero elimina uma posição ou apenas a sua contribuição?



Resposta individual. Explica o raciocínio antes de apresentar o resultado.

RESPOSTA: 1101_2 EM DECIMAL

Peso:	8	4	2	1
Bit:	1	1	0	1
Contribuição:	8	4	0	1

- O zero ocupa a posição de peso 2, mas acrescenta zero à soma.
- Mantemos todas as posições, mesmo quando o bit é 0.

$$1101_2 = 8 + 4 + 0 + 1 = 13_{10}$$

$$✓ \quad 13 = 8 + 4 + 1$$



Erro a evitar: começar com peso 2. A posição mais à direita tem sempre peso 1.

ALGORITMO: DECIMAL PARA BINÁRIO

Números inteiros não negativos

1 Dividir o número por 2 usando divisão inteira.

2 Registrar o resto: 0 ou 1.

3 Repetir com o quociente enquanto este for maior que zero.

4 Ler os restos do último para o primeiro.

$$13 = 2 \times 6 + 1$$

dividendo

divisor

quociente

resto

Dividendo		Divisão por 2		Quociente		Resto	
13	→	$13 \div 2$	→	6	→	1	← Primeiro resto (bit de menor peso)
6	→	$6 \div 2$	→	3	→	0	⋮
3	→	$3 \div 2$	→	1	→	1	↓
1	→	$1 \div 2$	→	0	→	1	← Último resto (bit de maior peso)

- O primeiro resto corresponde ao bit de menor peso.
- Cada divisão permite obter o bit seguinte.

Caso especial: Se o número inicial for 0, o resultado é 0_2 .

EXEMPLO: 13_{10} EM BINÁRIO

Divisão	Quociente	Resto
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

↑
Ler de
baixo
para
cima

- 1 O quociente de uma linha torna-se o número a dividir na seguinte.
- 2 Terminamos quando o quociente chega a 0.
- 3 A ordem de leitura dos restos é inversa à ordem em que foram obtidos.

$$13_{10} = 1101_2$$

Verificação:

$$1101_2 = 8 + 4 + 1 = 13_{10}$$

PERGUNTA: DECIMAL PARA BINÁRIO

$$22_{10} = ?_2$$

Aplica o algoritmo das divisões sucessivas por 2.

Número a dividir	Quociente	Resto
22		

- 1 Regista cada quociente e cada resto.
- 2 Identifica o momento em que deves parar.
- 3 Assinala a ordem de leitura dos restos.



Verifica a tua resposta convertendo o resultado de novo para decimal.

RESPOSTA: 22_{10} EM BINÁRIO

Divisão	Quociente	Resto
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1



Ler de
baixo
para cima

$$22_{10} = 10110_2$$

16	8	4	2	1
1	0	1	1	0

$$16 + 0 + 4 + 2 + 0 = 22$$



Ler os restos pela ordem em que aparecem daria uma representação incorreta.

SISTEMAS DE NUMERAÇÃO

O mesmo valor, três representações

DECIMAL • BASE 10

45₁₀

=

BINÁRIO • BASE 2

101101₂

=

HEXADECIMAL • BASE 16

2D₁₆

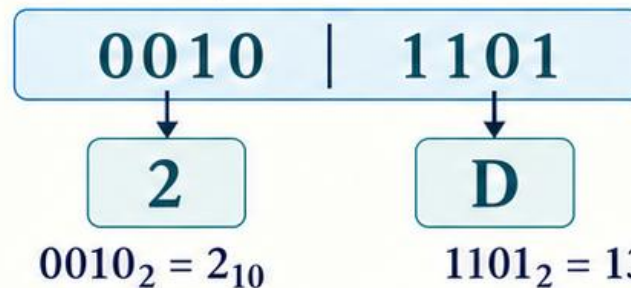
Como converter binário para hexadecimal (agrupamento de 4 bits)

1

Agrupar da direita para a esquerda em blocos de 4 bits.

2

Completar à esquerda com zeros (se necessário).



Cada algarismo hexadecimal representa exatamente 4 bits.

ALGORITMO: BINÁRIO PARA HEXADECIMAL

$16 = 2^4$: um algarismo hexadecimal corresponde a quatro bits.

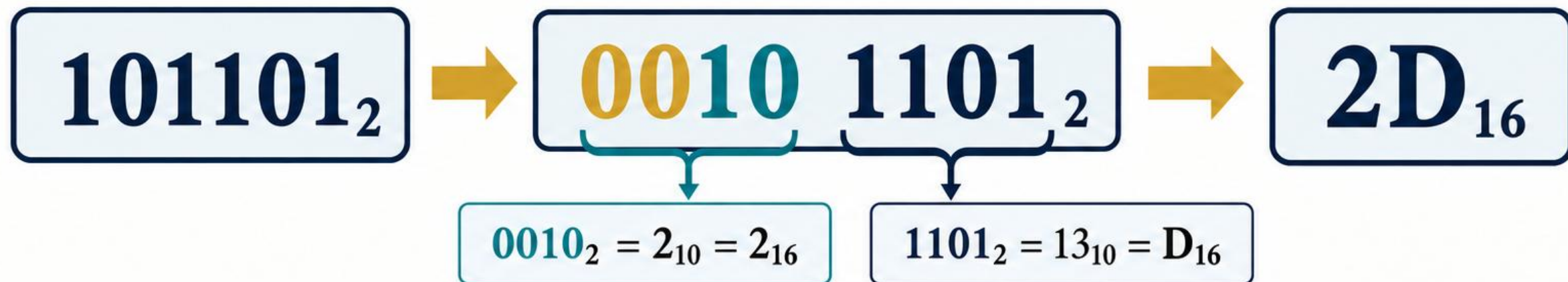
- 1 Agrupar os bits de quatro em quatro, começando pela direita.
- 2 Completar o grupo da esquerda com zeros, se necessário.
- 3 Converter cada grupo no algarismo hexadecimal correspondente.
- 4 Manter a ordem dos grupos.

0 0 1 0 1 1 0 1 0 0 1 1
└───┘ └───┘ └───┘
2 D 3

BINÁRIO	HEX	DECIMAL	BINÁRIO	HEX	DECIMAL
0000	0	0	1000	8	8
0001	1	1	1001	9	9
0010	2	2	1010	A	10
0011	3	3	1011	B	11
0100	4	4	1100	C	12
0101	5	5	1101	D	13
0110	6	6	1110	E	14
0111	7	7	1111	F	15

Agrupar pela direita preserva os pesos das posições.

EXEMPLO: 101101_2 EM HEXADECIMAL



- 1 Agrupar pela direita: $10 \mid 1101$.
- 2 Completar o primeiro grupo: $0010 \mid 1101$.
- 3 Substituir cada grupo: $2 \mid D$.

Zeros à esquerda
não alteram o valor.

Verificação: $2D_{16} = 2 \times 16 + 13 = 45_{10}$ | $101101_2 = 32 + 8 + 4 + 1 = 45_{10}$

PERGUNTA: BINÁRIO PARA HEXADECIMAL

$$111010_2 = ?_{16}$$

- 1 Agrupa os bits pela direita.
- 2 Acrescenta apenas os zeros necessários à esquerda.
- 3 Converte cada grupo e mantém a ordem.

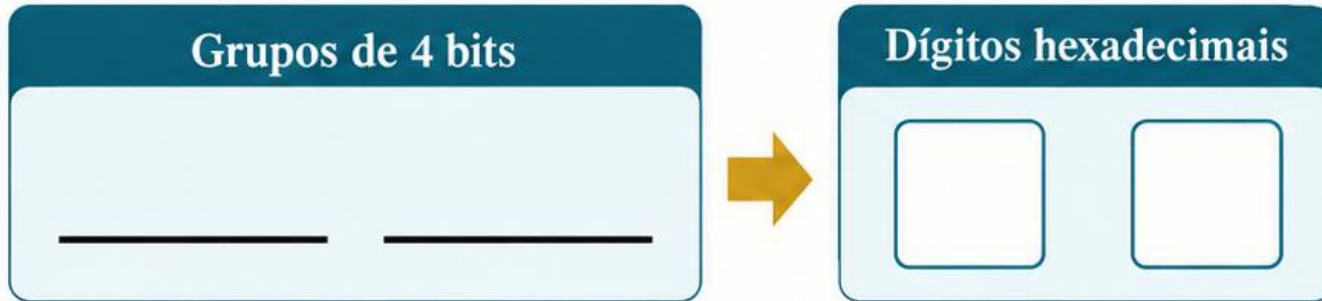


Tabela de correspondência

Binário (4 bits)	Hexadecimal	Binário (4 bits)	Hexadecimal
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F



Justifica por que podes acrescentar zeros à esquerda.

RESPOSTA: 111010_2 EM HEXADECIMAL

$$111010_2 = \underbrace{0011}_{\text{light blue}} \underbrace{1010}_{\text{yellow}}_2 = 3A_{16}$$

↓

$$\begin{array}{c} 0011_2 \\ = 2 + 1 = 3 \end{array}$$

↓

$$\begin{array}{c} 1010_2 \\ = 8 + 2 = 10 = A_{16} \end{array}$$

- O agrupamento começa pela direita.
- Os dois zeros acrescentados à esquerda não alteram o valor.
- A ordem dos grupos mantém-se: primeiro 3, depois A.

VERIFICAÇÃO

$$3A_{16} = 3 \times 16 + 10 = 58_{10}$$

$$111010_2 = 32 + 16 + 8 + 2 = 58_{10}$$

ASCII E UNICODE

ASCII

128 códigos · 7 bits

Inclui letras latinas básicas, algarismos, sinais e códigos de controlo.

Carácter	Decimal	Binário em 8 bits
A	65	01000001
B	66	01000010
C	67	01000011

Nos exemplos, o bit inicial 0 completa um byte.



Limitação

Não inclui letras como 'ã' nem muitos outros sistemas de escrita.

UNICODE

Um repertório muito mais abrangente

A ã Ω 中 €

Atribui pontos de código a caracteres de muitos idiomas e símbolos.

'A' → U+0041

UTF-8 é uma forma de codificar Unicode em bytes.

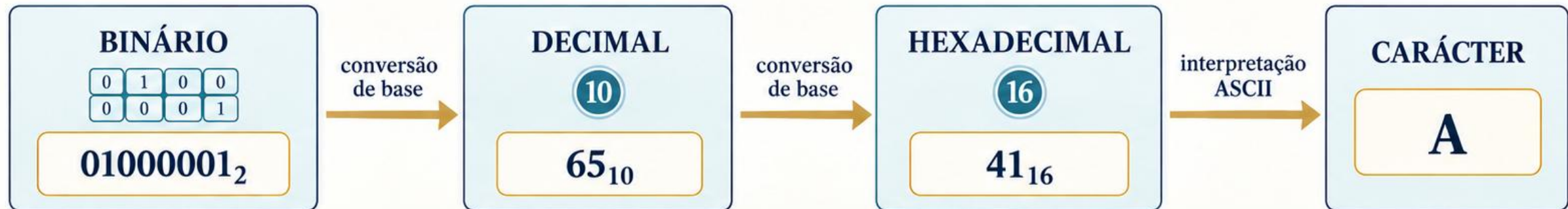


Em UTF-8, os caracteres ASCII mantêm os mesmos valores de um byte.



Unicode identifica os caracteres; uma codificação como **UTF-8** define os bytes.

DOS BITS AOS CARACTERES



CÁLCULO EM DECIMAL

O valor decimal resulta da soma dos pesos dos bits que valem 1.

$$64 + 1 = 65$$

(bits nas posições 2^6 e 2^0)

CÁLCULO EM HEXADECIMAL

O valor hexadecimal resulta da soma dos dígitos com o peso da base 16.

$$4 \times 16 + 1 = 65$$

(dígitos 4 e 1 nas posições 16^1 e 16^0)

INTERPRETAÇÃO COMO CARÁCTER

Em ASCII, o código 65 corresponde à letra 'A'.



Importante:

A letra 'A' não é uma base numérica: resulta da interpretação do código.



O padrão 01000001 representa sempre uma letra?

Não. Pode representar o número 65 ou o carácter 'A', conforme o contexto.

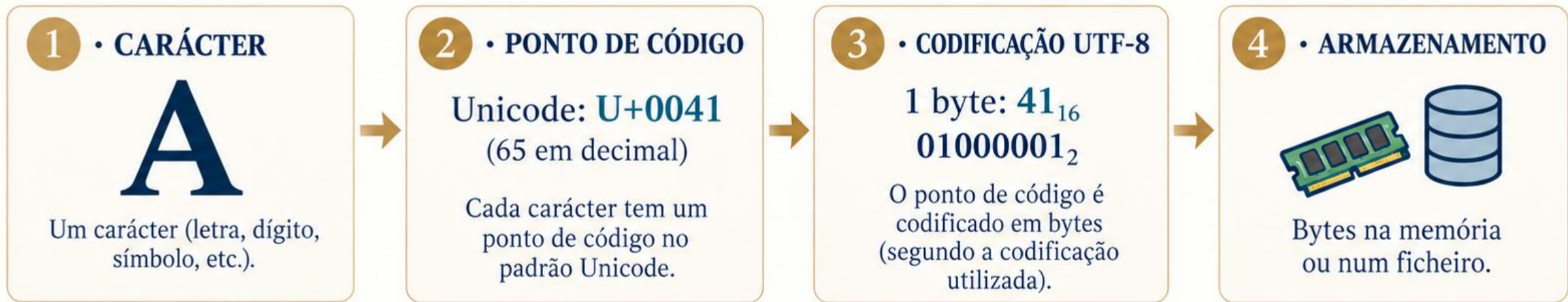


IDEIA-CHAVE

Converter a base mantém o valor; descodificar atribui significado.

ARMAZENAMENTO DE CARACTERES

Da letra aos bytes – e dos bytes à letra



← **Leitura:** os bytes são descodificados segundo a codificação usada.

Exemplo em UTF-8: **AB**

0	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

A • 65

2 caracteres ASCII → 2 bytes de conteúdo.

0	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

B • 66



Nem todo o carácter ocupa 1 byte.

Em UTF-8, 'ã' ocupa 2 bytes.

A codificação tem de ser conhecida para interpretar os dados corretamente.



O computador armazena **bits**; a codificação permite **recuperar o texto**.

ADIÇÃO BINÁRIA DE INTEIROS POSITIVOS

REGRAS DA ADIÇÃO BINÁRIA

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10_2$$

Escrever 0 e transportar 1 para a coluna seguinte.

Com transporte:

$$1 + 1 + 1 = 11_2.$$

Escrever 1 e transportar 1.

EXEMPLO DE SOMA (8 BITS)

$$\begin{array}{r} 00000101 +5 \\ + 00000011 +3 \\ \hline 00001000 +8 \end{array}$$

DA DIREITA PARA A ESQUERDA

Peso 1 (2^0)

$$1 + 1 = 10_2.$$

Escrever 0,
transportar 1.



Peso 2 (2^1)

$$0 + 1 + 1 = 10_2.$$

Escrever 0,
transportar 1.



Peso 4 (2^2)

$$1 + 0 + 1 = 10_2.$$

Escrever 0,
transportar 1.



Peso 8 (2^3)

$$0 + 0 + 1 = 1.$$

Escrever 1.

Alinhar os bits, somar pela direita e considerar o transporte.

EXERCÍCIO: SOMAR 7 E 6

Inteiros positivos representados com 8 bits

Transportes: ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐

	0	0	0	0	0	1	1	1	→ 7
+	0	0	0	0	0	1	1	0	→ 6
	?	?	?	?	?	?	?	?	

- 1 Soma da direita para a esquerda.
- 2 Assinala cada transporte.
- 3 Escreve o resultado com 8 bits.
- 4 Converte o resultado para decimal e verifica.



Justifica o que acontece quando a soma de uma coluna é 2 ou 3.

RESPOSTA: 7 + 6

Transportes: 0 0 0 0 **1** **1** 0 0

$$\begin{array}{r} 00000111 \\ + 00000110 \\ \hline 00001101 \end{array} \begin{array}{l} +7 \\ +6 \\ +13 \end{array}$$

Peso 1: $1 + 0 = 1$.

Peso 2: $1 + 1 = 10_2$. Escrever 0, transportar 1.

Peso 4: $1 + 1 + 1 = 11_2$. Escrever 1, transportar 1.

Peso 8: $0 + 0 + 1 = 1$.



O transporte soma-se aos dois bits da coluna seguinte.

Verificação: $00001101_2 = 8 + 4 + 1 = 13_{10}$

ENTRADA, PROCESSAMENTO E SAÍDA

ENTRADA

número inteiro não negativo,
escrito em decimal.

Exemplo: 6.



PROCESSAMENTO

dividir por 2, recolher os restos
e construir a representação
pela ordem correta.

		Resto
6	÷ 2	0
3	÷ 2	1
1	÷ 2	1
0		

A yellow arrow points from the 'Resto' column (0, 1, 1) to the output stage, indicating the order of bits from bottom to top.

SAÍDA

sequência de bits.

Exemplo: 110₂.



Dados:

aquilo que o algoritmo recebe.

Operações:

aquilo que o algoritmo faz.

Resultado:

aquilo que o algoritmo apresenta.

6₁₀ e 110₂ representam a mesma quantidade.

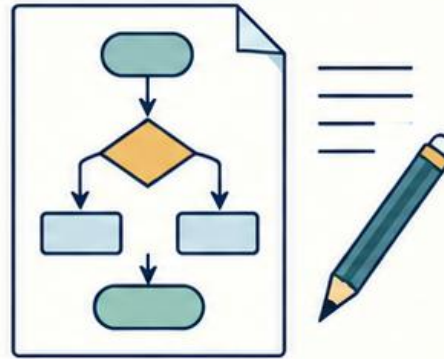
CONCEBER ANTES DE PROGRAMAR

1 COMPREENDER



- Qual é o problema?
- Que dados existem?
- Que resultado se pretende?

2 CONCEBER



- Definir entradas, operações, saídas e passos.

3 PROGRAMAR



- Traduzir a solução para uma linguagem.

ANÁLISE DO PROBLEMA

- ✓ Objetivo
- ✓ Dados
- ✓ Restrições
- ✓ Casos especiais
- ✓ Critério de sucesso



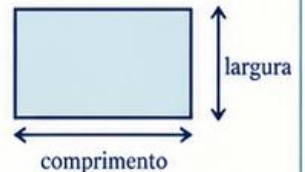
**Uma solução correta
para o problema errado
continua a ser inútil.**

EXEMPLO: ÁREA DE UM RETÂNGULO

Dados: comprimento e largura.

Resultado: área.

Restrição: dimensões positivas.

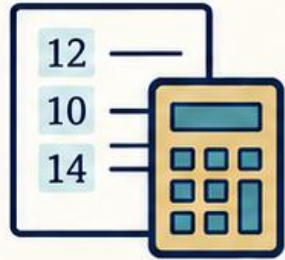


IDEIA-CHAVE: Primeiro definimos a solução; depois escrevemos o código.

DECOMPOR E CONSTRUIR UM ALGORITMO

PROBLEMA

Calcular a média de três classificações e indicar a situação final.



DECOMPOSIÇÃO EM TAREFAS MAIS SIMPLES

- 1  Ler três notas
- 2  Somar as notas
- 3  Dividir por 3
- 4  Comparar a média com 10
- 5  Determinar Aprovado/Reprovado
- 6  Apresentar média e situação

ASSUNÇÃO DO EXEMPLO

Neste exemplo, aprovação se $\text{média} \geq 10$; notas entre 0 e 20, com igual peso.

EXEMPLO

12, 10 e 14 $\rightarrow$ média 12 $\rightarrow$ **Aprovado.**

ENTRADAS

nota1, nota2, nota3

PROCESSAMENTO

$\text{média} = (\text{nota1} + \text{nota2} + \text{nota3}) / 3$

SAÍDAS

média e situação



Decomp é dividir um problema em tarefas mais simples e ordenadas.

CARACTERÍSTICAS DE UM ALGORITMO

1

RIGOROSO

Instruções precisas, sem ambiguidade.



“Misturar até ficar suficiente”



Ambígua



“Misturar durante 30 segundos”

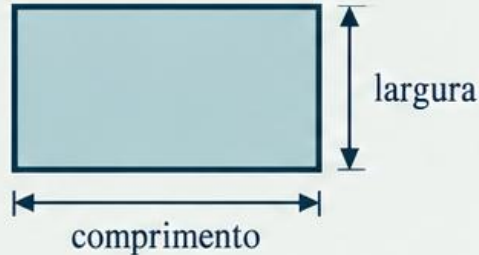


Objetiva

2

EFICAZ

Produz o resultado pretendido para os dados admissíveis.



Área =
comprimento
× largura

3

FINITO

Termina após um número finito de passos.



IDEIA-CHAVE

Um algoritmo é uma sequência finita e bem definida de passos para resolver um problema.

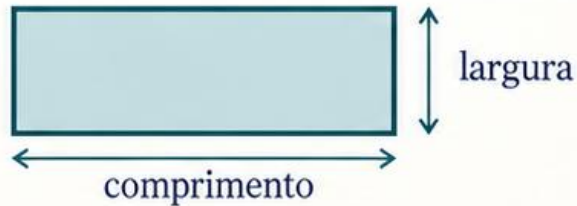


Os passos devem ser executáveis e estar ordenados.

QUESTÕES – PROBLEMAS E ALGORITMOS

1 ORDENAR

Para calcular o perímetro de um retângulo, ordene:



- A Apresentar o perímetro
- B Ler comprimento e largura
- C Calcular $P = 2 \times (\text{comprimento} + \text{largura})$

2 DECOMPOR

Um veículo percorreu 300 km e consumiu 18 L. Identifique entradas, cálculo e saída para obter o consumo em L/100 km.



3 AVALIAR

A instrução «misturar até ficar suficiente» é rigorosa?

Justifique e proponha uma alternativa precisa.



Trabalhe a pares. Explique o raciocínio antes de apresentar o resultado.

RESPOSTAS — PROBLEMAS E ALGORITMOS

1 ORDEM CORRETA

B → C → A

Ler dimensões → calcular perímetro → apresentar resultado.



B

Ler dimensões



C

Calcular perímetro



A

Apresentar resultado

Exemplo:

Comprimento 5 e largura 4:

$$P = 2 \times (5 + 4) = 18$$

2 CONSUMO MÉDIO

Entradas: distância = 300 km;
combustível = 18 L.



300 km
(distância)



18 L
(combustível)

$$\text{Consumo} = \frac{\text{litros}}{\text{quilômetros}} \times 100$$

$$18 / 300 \times 100 = \mathbf{6 \text{ L/100 km}}$$



A distância tem de ser maior que zero.

3 RIGOR

Não. «Suficiente» não define um critério objetivo.



«Suficiente»
é ambíguo.



Alternativa:
«Misturar durante
30 segundos».



Ideia-chave: Definir os dados, ordenar os passos e eliminar ambiguidades.

ALGORITMO, PSEUDOCÓDIGO E PROGRAMA

1 ALGORITMO

A solução lógica



Obter dimensões



Multiplicar



Apresentar área

2 PSEUDOCÓDIGO

Descrição estruturada da solução

Exemplo – Área de um retângulo

Início

Ler comprimento

Ler largura

area ← comprimento × largura

Escrever area

Fim

3 PROGRAMA

Implementação numa linguagem

Exemplo – Em Python

```
comprimento = float(input('Comprimento: '))  
largura = float(input('Largura: '))  
area = comprimento * largura  
print(area)
```



O pseudocódigo não é executado diretamente pelo computador.
Permite pensar na lógica sem depender da sintaxe de uma linguagem.



Problema



Algoritmo



Pseudocódigo

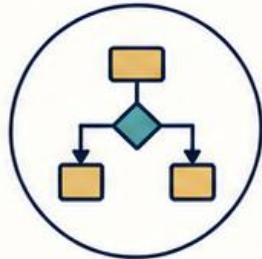


Programa

ESCREVER PSEUDOCÓDIGO



Claro



Ordenado



Rigorouso

INSTRUÇÕES FUNDAMENTAIS

INÍCIO / FIM

delimitar o algoritmo

LER

obter dados



atribuir um valor

ESCREVER

apresentar resultados

EXEMPLO: MÉDIA DE TRÊS CLASSIFICAÇÕES

Início

Ler nota1, nota2, nota3

} Entrada de dados

soma ← nota1 + nota2 + nota3

media ← soma / 3

} Processamento

Escrever media

} Saída de resultados

Fim

EXEMPLO DE EXECUÇÃO

12, 10, 14 → soma 36 → média 12



Cada variável deve receber um valor antes de ser utilizada num cálculo.

QUESTÕES — PSEUDOCÓDIGO

1

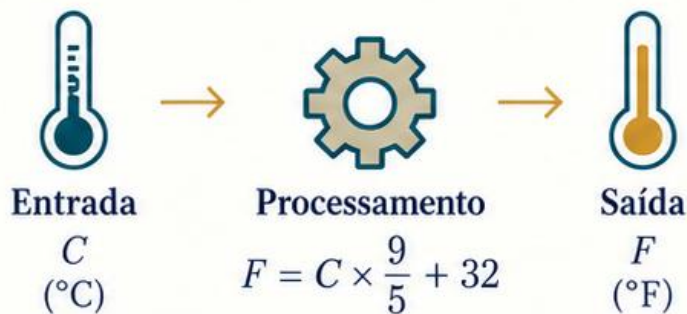
TEMPERATURA

Use a fórmula

$$F = C \times \frac{9}{5} + 32$$

Identifique a entrada, o processamento e a saída.

Qual é o resultado para $C = 20$?



2

COMPRA

Escreva pseudocódigo que:

1. Leia o preço.
2. Leia a quantidade.
3. Calcule o total.
4. Apresente o resultado.

Teste o seu pseudocódigo com:

- preço 4 €
- quantidade 3.



3

ENCONTRAR O ERRO

Considere o seguinte pseudocódigo:

```
Início  
  area ← comprimento × largura  
  Ler comprimento  
  Ler largura  
  Escrever area  
Fim
```

O que está errado na ordem das instruções? Corrija.



Antes de calcular,
confirme que os dados
necessários já foram lidos.

RESPOSTAS — PSEUDOCÓDIGO

1 • TEMPERATURA



Entrada:

C (graus Celsius)



Processamento:

$$F \leftarrow C \times \frac{9}{5} + 32$$



Saída:

F (graus Fahrenheit)



$20^{\circ}\text{C} \rightarrow 68^{\circ}\text{F}$

2 • COMPRA

Início

Ler preço, quantidade

$\text{total} \leftarrow \text{preço} \times \text{quantidade}$

Escrever total

Fim



4 €



3

= 12 €

3 • ORDEM CORRETA

Início

Ler comprimento

Ler largura

$\text{area} \leftarrow \text{comprimento} \times \text{largura}$

Escrever area

Fim



IDEIA-CHAVE

Os dados têm de ser obtidos antes de serem usados.

TESTAR, CORRIGIR E APERFEIÇOAR



ERRO DE SINTAXE

Viola as regras da linguagem.

Exemplo (Python):

```
print(area
```



ERRO LÓGICO

O programa pode executar, mas calcula o resultado errado.

Exemplo (Python):

```
area = comprimento + largura
```

EXEMPLO: Área de um retângulo.

Dados	Esperado	Obtido com +	Conclusão
5 e 4	20	9	Erro lógico

Correção:

```
area = comprimento * largura
```

COMO TESTAR O PROGRAMA?

- ✓ Testar valores simples.
- ✓ Comparar com cálculo manual.
- ✓ Considerar casos-limite.
- ✓ Verificar se termina.
- ✓ Confirmar que resolve o problema.



IDEIA-CHAVE

Testar é comparar o resultado produzido com o resultado esperado.

EXERCÍCIO INTEGRADO – CUSTO DA VIAGEM

PROBLEMA



Calcular o custo do combustível de uma viagem.

DADOS CONHECIDOS



Distância:
150 km



Consumo médio:
6 L/100 km



Preço do combustível:
1,80 €/L

TAREFAS

1



Identifique
entradas e saída.

2



Decomponha
o cálculo em
duas etapas.

3



Escreva
o pseudocódigo.

4



Calcule e verifique
o resultado.



PERGUNTA

Que quantidade deve calcular
antes do custo?



ASSUNÇÕES

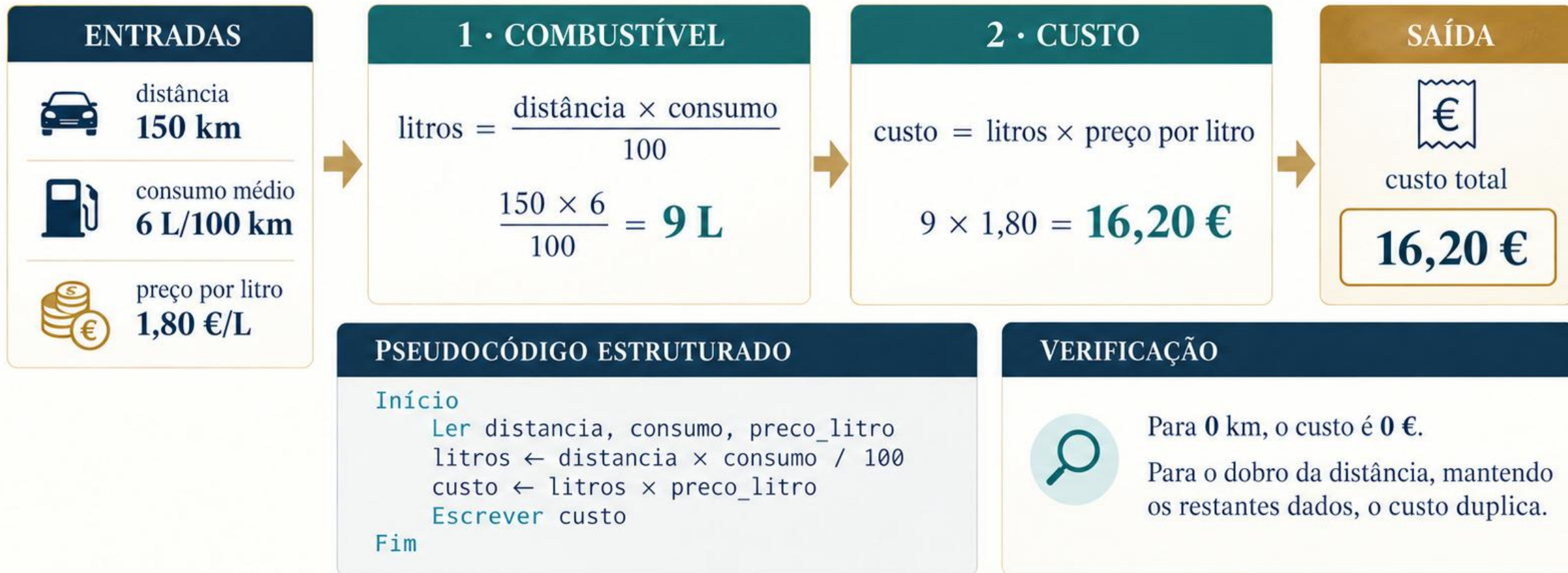
- Consumo constante.
- Sem portagens nem outros custos.
- Dados não negativos.



IDEIA-CHAVE

Explique a sequência de passos
e mantenha as unidades.

RESPOSTA – CUSTO DA VIAGEM



Compreender



decompor



ordenar



representar



testar.

PSEUDOCÓDIGO: DECIMAL PARA BINÁRIO

INÍCIO

LER número

binário = ""

SE número = 0 **ENTÃO**

binário = "0"

SENÃO

ENQUANTO número > 0

resto = número **MOD** 2

binário = texto(resto) + binário

número = número **DIV** 2

FIM ENQUANTO

FIM SE

ESCREVER binário

FIM



Pré-condição: número inteiro não negativo.



DIV: quociente da divisão inteira.
 $6 \text{ DIV } 2 = 3$.



MOD: resto da divisão inteira.
 $3 \text{ MOD } 2 = 1$.



texto(resto) + binário: junta o novo bit à esquerda da sequência.



ENQUANTO: repete até número ser 0.

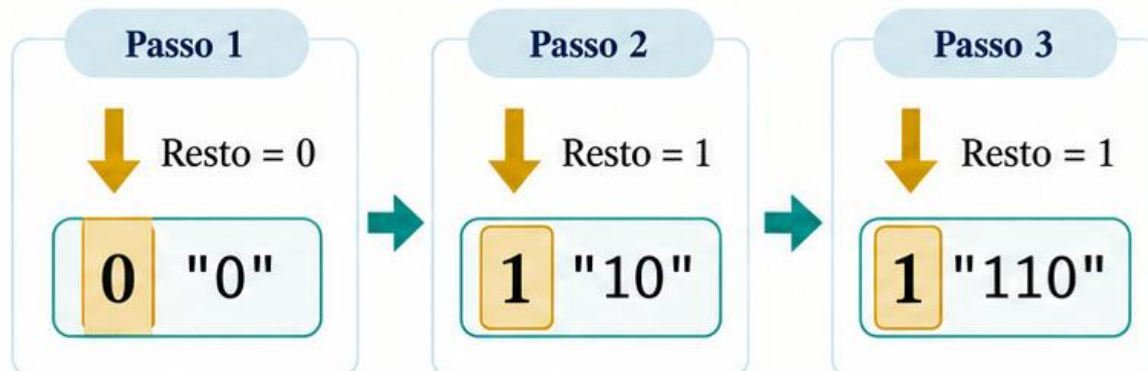


O caso inicial 0 precisa de tratamento próprio.

EXECUÇÃO GUIADA: CONVERTER 6_{10}

Estado inicial:
número = 6; binário = ""

Passo	Número antes	Resto (MOD 2)	Binário após juntar à esquerda	Número depois (DIV 2)
1	6	0	"0"	3
2	3	1	"10"	1
3	1	1	"110"	0



- Cada resto novo entra à esquerda.
- Quando número = 0, a condição *número* > 0 é falsa.
- A repetição termina e o algoritmo escreve 110.

$$6_{10} = 110_2$$

Verificação:

$$4 + 2 + 0 = 6$$

PERGUNTA FINAL: DUAS CONVERSÕES

$$26_{10} = ?_2 = ?_{16}$$



Em pares, resolve e explica o método.

1

DECIMAL PARA BINÁRIO:

Regista as divisões por 2, os quocientes e os restos.

2

BINÁRIO PARA HEXADECIMAL:

Agrupa o resultado em conjuntos de quatro bits.

3

VERIFICAÇÃO:

Converte a representação hexadecimal para decimal.

?

O valor do número mudou em alguma etapa?
Justifica.



RESPOSTA FINAL: O NÚMERO 26

1 Conversão de decimal para binário

Divisão sucessiva por 2, registrando os restos.

Divisão	Quociente	Resto
$26 \div 2$	13	0
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

Ler os restos
de baixo
para cima:

11010_2

2 Conversão de binário para hexadecimal

Agrupar os bits de quatro em quatro, da direita para a esquerda.

$$11010_2 = 0001 \ 1010_2$$

$$0001 = 1$$

$$1010 = A$$

3 Verificação em hexadecimal

$$1A_{16} = 1 \times 16 + 10 = 26_{10}$$

4 Equivalência final

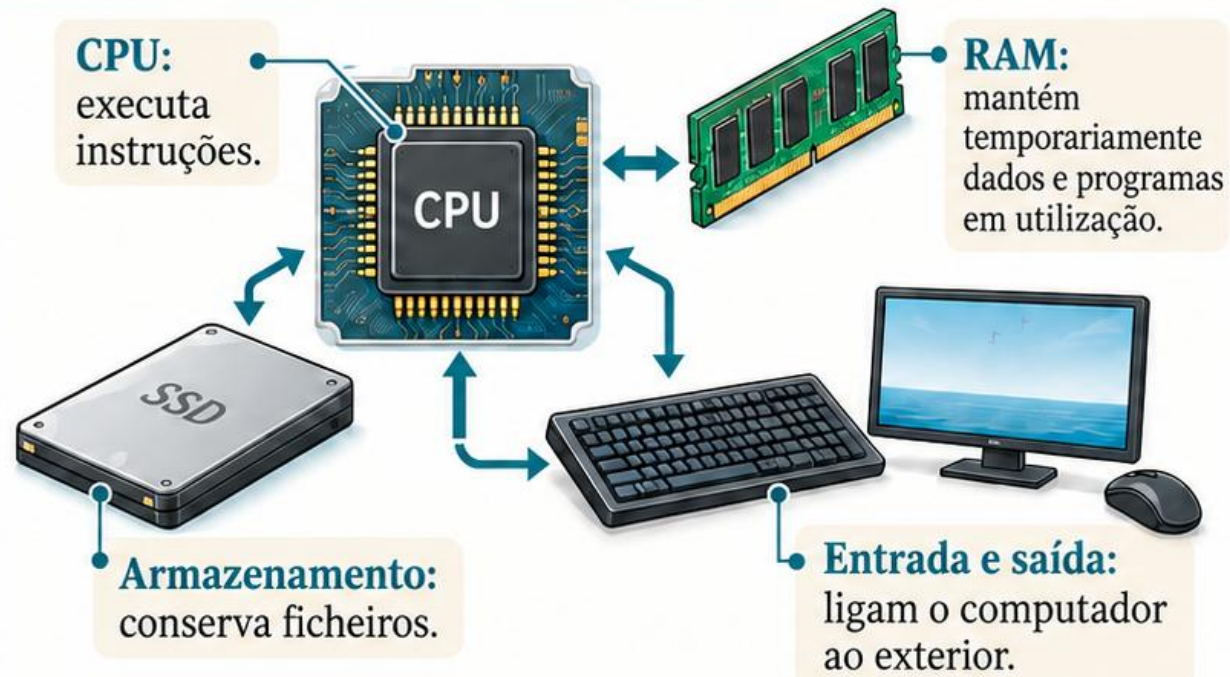
$$26_{10} = 11010_2 = 1A_{16}$$

5 Conclusão

O valor mantém-se. Apenas a representação muda.

SÍNTESE DA AULA

ARQUITECTURA DO COMPUTADOR



Um **algoritmo** define os passos. $\longrightarrow$ Um **programa** implementa esses passos numa linguagem.

Muda a representação. Mantém-se o valor.

$$26_{10} = 11010_2 = 1A_{16}$$

ALGORITMOS E REPRESENTAÇÕES

1

BINÁRIO PARA DECIMAL:

multiplicar bits pelos pesos e somar.

1	1	0	1	0
↓	↓	↓	↓	↓
2^4	2^3	2^2	2^1	2^0

2

DECIMAL PARA BINÁRIO:

dividir por 2 e ler os restos pela ordem inversa.

n	$\div 2$	$\rightarrow$	resto
	$\div 2$	$\rightarrow$	resto
	$\div 2$	$\rightarrow$	resto
	...		

3

BINÁRIO PARA HEXADECIMAL:

agrupar quatro bits pela direita e converter cada grupo.

0001	1010
↓	↓
1	A